

WeeFly · MVP 2 · Guia de deploy no EC2

Para	Fábio · Sarin
Data	30 de setembro de 2026
O que vai para o ar	<code>main</code> a partir do <code>f0aac44</code> (passos 4 a 9 do MVP 2). Hoje corre o <code>06c4b44</code>
Servidor	<code>i-0152d6578ff6a5801</code> · <code>eu-west-1</code> (Irlanda) · <code>concierge.weefly.africa</code>
Aplicação	<code>/var/www/concierge</code> · PM2 <code>weefly-concierge</code> , do utilizador <code>ubuntu</code>
Base de dados	Migrações <code>0026</code> a <code>0030</code> já aplicadas . Não há SQL a correr neste deploy

Cada bloco de comandos é para colar inteiro na sessão. Depois de cada passo há um **Confirma:** — se não bater, pára aí e não avances.

0 · Antes de começar

1. **Snapshot do disco.** EC2 → *Volumes* → o volume da instância → *Create snapshot*, com a descrição `antes-mvp2-f0aac44`. Se o deploy correr mal, é daqui que se volta em minutos. 2. **Hora.** Faz o deploy fora de horas de trabalho em Cabo Verde. O `pm2 reload` corta durante 2 a 5 segundos, e o build ocupa a máquina uns minutos.

1 · Entrar no servidor

Consola AWS → região `eu-west-1` → EC2 → *Instances* → `i-0152d6578ff6a5801` → **Connect** → **Session Manager** → *Connect*.

O Session Manager entra como `ssm-user`. A aplicação, o PM2 e o git são todos do `ubuntu`, por isso muda logo de utilizador:

```
sudo -iu ubuntu
whoami
```

Confirma: `ubuntu`.

> **Nunca** `sudo pm2 ...`. Nem um `sudo pm2 list`. O PM2 é por utilizador: com `sudo` arranca um segundo PM2, vazio, em `/root/.pm2`, e parece que a aplicação desapareceu. Os `sudo` deste guia são só para `systemctl`, `nginx` e `/etc`. O `ubuntu` tem `sudo` sem password.

2 · Ver o que está no ar

```
cd /var/www/concierge
git log --oneline -1
git status -sb
pm2 status
```

Confirma:

- `git log` mostra `06c4b44` (ou outro commit anterior ao `f0aac44`).
- `git status` diz `## main...origin/main`, sem ficheiros alterados. Se houver algum `M` ou `??`, alguém mexeu no servidor à mão. Não apagues nada: manda-me a lista.
- `pm2 status` mostra `weefly-concierge` como `online`.

3 · Ver o ambiente

As variáveis estão em `/var/www/concierge/.env.production`. Este comando mostra só os nomes e os valores que não são segredos:

```
cd /var/www/concierge
ls -l .env.production
grep -E '^(NEXT_PUBLIC_SITE_URL|SERVED_HOSTS)= ' .env.production
grep -q '^PC_CRON_TOKEN=' .env.production && echo "PC_CRON_TOKEN: existe" || echo "PC_CRON_T
```

Confirma:

LINHA	TEM DE SER
<code>ls -l</code>	<code>-rw-----</code> (só o <code>ubuntu</code> lê)
<code>NEXT_PUBLIC_SITE_URL</code>	<code>https://concierge.weefly.africa</code> . Se disser <code>duckdns</code> , corrija antes do build: é o endereço que vai dentro de todos os emails
<code>SERVED_HOSTS</code>	<code>concierge.weefly.africa</code> . Se a linha não existir, a aplicação responde a qualquer endereço (funciona, mas o endereço antigo continua a abrir)
<code>PC_CRON_TOKEN</code>	<code>existe</code>

Para acrescentar ou corrigir: `nano .env.production`, gravar com `Ctrl+O`, `Enter`, sair com `Ctrl+X`.

Se o `PC_CRON_TOKEN` **faltar**, cria-o agora. Precisa do build e do reload do passo 4 para a aplicação o ler:

```
cd /var/www/concierge
echo "PC_CRON_TOKEN=$(openssl rand -hex 32)" >> .env.production
chmod 600 .env.production
```

> As variáveis `NEXT_PUBLIC_...` são lidas **no build**. Mudar uma obriga a correr o passo 4 inteiro, não só o `pm2 reload`.

4 · Deploy

```
cd /var/www/concierge
git pull --ff-only origin main
git log --oneline -3

npm ci
npm run build

pm2 reload weefly-concierge --update-env
pm2 save
```

Confirma:

- O `git log` mostra no topo `e3320a0` ou mais recente, com o `f0aac44` logo abaixo.
- O `npm run build` acaba com a lista das rotas. Têm de aparecer `/m/[org]/[token]`, `/agente/financas`, `/gestao/numeros` e `/gestao/casos`.
- A seguir:

```
pm2 status
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:3000/login
pm2 logs weefly-concierge --lines 30 --nostream
```

`online` com `0` reinícios desde o reload, `200`, e nenhum `Error` nos registos.

Se o `git pull` recusar (`Not possible to fast-forward`): há alterações locais no servidor. Não forces — manda-me o `git status`.

Se o `npm run build` morrer sem mensagem (`Killed`), faltou memória. O `next build` chega a pedir 1,5 GB. Liga swap uma vez e repete o `npm run build`:

```
free -h
sudo falldate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

Enquanto o build não acabar, a versão antiga continua a correr. O `pm2 reload` só a troca no fim.

5 · Os alertas diários

Um timer do systemd chama a aplicação todos os dias às 08:00 de Cabo Verde. Faz duas coisas: o **alerta de saldo** das bolsas (`PAR-05`) e o **aviso de passaporte a expirar** (`DAT-02`). Sem este passo, nenhum dos dois sai.

O timer precisa do mesmo `PC_CRON_TOKEN` que a aplicação, num ficheiro que só o root lê:

```
cd /var/www/concierge
TOKEN=$(grep '^PC_CRON_TOKEN=' .env.production | cut -d= -f2-)
sudo mkdir -p /etc/weefly
echo "PC_CRON_TOKEN=$TOKEN" | sudo tee /etc/weefly/pc-cron.env > /dev/null
sudo chmod 600 /etc/weefly/pc-cron.env
unset TOKEN

sudo cp deploy/systemd/weefly-b2g-alerts.* deploy/systemd/weefly-pc-expire.* /etc/systemd/sy
sudo systemctl daemon-reload
sudo systemctl enable --now weefly-b2g-alerts.timer weefly-pc-expire.timer
```

O `weefly-pc-expire` (fecha os pagamentos fora de prazo) vai no mesmo bloco. Se já estava ligado, repetir não faz mal.

Corre os dois uma vez à mão:

```
sudo systemctl start weefly-b2g-alerts.service weefly-pc-expire.service
systemctl status weefly-b2g-alerts.service weefly-pc-expire.service --no-pager | grep -E 'Lo
systemctl list-timers | grep weefly
```

Confirma: os dois serviços com `status=0/SUCCESS`, e os dois timers com a próxima execução marcada.

SE APARECER

QUER DIZER

`curl: (22) ... 404` A aplicação não tem o `PC_CRON_TOKEN`: falta no `.env.production`, ou o passo 4 não correu depois de o acrescentar

`curl: (22) ... 401` O token de `/etc/weefly/pc-cron.env` é diferente do da aplicação. Repete o primeiro bloco deste passo

6 · Confirmar de fora

Estes pedidos testam o endereço público, pelo NGINX. Podem correr na sessão ou num terminal teu:

```
for p in /login /gestao/numeros /gestao/casos /agente/financas /m/nao-existe/aaaaaaaaaaaaaaaaa
printf "%-45s %s\n" "$p" "$(curl -s -o /dev/null -w '%{http_code}' https://concierge.weefl
done
```

Confirma:

ENDEREÇO

CÓDIGO PORQUÊ

<code>/login</code>	<code>200</code>	A aplicação está no ar
<code>/gestao/numeros</code> · <code>/gestao/casos</code> · <code>/agente/financas</code>	<code>307</code>	Sem sessão, manda para o login

ENDEREÇO	CÓDIGO	PORQUÊ
<code>/m/nao-existe/...</code>	404	Um ministério que não existe não abre
<code>/api/finance/export</code>	404	A exportação não responde sem sessão
<code>/api/b2g/alerts</code>	401	O cron não corre sem o token

Depois, **no browser**, com uma conta Admin WeeFly: *Admin > Números*, *Admin > Casos* e *Admin > B2G* abrem, e no Agente aparece *Finanças* (esta só para um Admin do parceiro).

7 · Desligar o endereço antigo (se ainda não foi feito)

Vem da primeira entrega (`MIG-02`). Confirma primeiro se já está feito:

```
curl -s -o /dev/null -w "%{http_code}\n" http://weefly.duckdns.org/login
```

Se der `000` (ligação fechada), já está. Se der `200` ou `301`, instala:

```
cd /var/www/concierge
sudo rm -f /etc/nginx/sites-enabled/weefly-duckdns-redirect.conf /etc/nginx/sites-available/
sudo cp deploy/nginx/weefly-duckdns-off.conf /etc/nginx/sites-available/
sudo ln -sf /etc/nginx/sites-available/weefly-duckdns-off.conf /etc/nginx/sites-enabled/
grep -rn "duckdns" /etc/nginx/sites-enabled/ | grep server_name
sudo nginx -t && sudo systemctl reload nginx
```

O `grep` só pode mostrar o ficheiro `weefly-duckdns-off.conf`. Se mostrar outro, tira `weefly.duckdns.org` do `server_name` desse ficheiro (`sudo nano ...`) antes do `nginx -t`.

Depois, desliga o actualizador do DuckDNS, ou apaga o domínio na conta DuckDNS.

Já agora, o limite de upload: `grep -rn client_max_body_size /etc/nginx/sites-enabled/` tem de mostrar `12M`. Sem ele, os comprovativos acima de 1MB falham com `413`.

8 · Voltar atrás

As migrações só acrescentaram tabelas e colunas: **o código antigo funciona com a base de dados nova**. Voltar atrás é só código, e não é preciso SQL.

```
cd /var/www/concierge
git checkout 06c4b44
npm ci && npm run build
pm2 reload weefly-concierge --update-env
```

Para voltar ao `main` depois: `git checkout main && git pull --ff-only`, e repetir o build e o reload.

Se até isso falhar, o snapshot do passo 0 repõe a máquina inteira.

9 · Problemas conhecidos

SINTOMA	CAUSA	O QUE FAZER
<code>pm2 status</code> vazio	Correste o <code>pm2</code> sem ser como <code>ubuntu</code> , ou com <code>sudo</code>	<code>exit</code> até ao <code>ssm-user</code> , <code>sudo -iu ubuntu</code> , <code>pm2 status</code>
Tudo dá <code>404</code> , até o <code>/login</code>	<code>SERVED_HOSTS</code> não tem o endereço por onde estás a entrar	Corrige no <code>.env.production</code> e faz <code>pm2 reload weefly-concierge -update-env</code>
Os emails levam links para o duckdns	<code>NEXT_PUBLIC_SITE_URL</code> errado no build	Corrige e repete o passo 4 inteiro
Build <code>Killed</code>	Memória	Swap (passo 4)
Upload do comprovativo falha com <code>413</code>	<code>client_max_body_size</code> do NGINX	<code>docs/deploy-ec2.md</code> , secção 1
O cron dá <code>404</code> ou <code>401</code>	O token	Passo 5

10 · Depois do deploy

Com o deploy feito, correm os testes. Estão no documento de estado (`WeeFly_MVP2_Estado_2026-09-30_v2`): os Blocos A, B e D, e mais os das finanças, da intervenção do Admin e das fichas dos viajantes.

Nenhum link vai para um ministério real antes de os testes do Bloco D passarem e de haver resposta a L1, L2 e L3.

Continua pendente, e fica com o Sarin:

- fechar a porta 22 (o acesso já é pelo Session Manager);
- DNS `*.weefly.africa` e certificado wildcard (`TEN-04`).